



PROYECTO INTEGRADOR

UAJS SMART CAMPUS

PLATAFORMA DISTRIBUIDA PARA LA GESTIÓN Y ARTICULACIÓN DE SERVICIOS UNIVERSITARIOS

1. DESCRIPCIÓN DEL PROYECTO

La Universidad Antonio José de Sucre cuenta con diferentes servicios digitales orientados a facilitar los procesos académicos y administrativos de estudiantes, docentes y demás miembros de la comunidad universitaria. A partir de este contexto, se propone el desarrollo de UAJS Smart Campus, una plataforma web concebida como un sistema distribuido de servicios universitarios, cuyo propósito será integrar y facilitar el acceso a diferentes funcionalidades institucionales desde una interfaz unificada.

El proyecto será desarrollado inicialmente como una aplicación web utilizando HTML5, CSS3, JavaScript y React, aplicando una arquitectura estructurada bajo el patrón Modelo-Vista-Controlador (MVC).

Posteriormente, la solución evolucionará hacia una arquitectura basada en API REST y servicios independientes, permitiendo a los estudiantes aplicar los conceptos propios de la asignatura de Sistemas Distribuidos.



La propuesta no pretende sustituir los sistemas institucionales existentes. Su propósito académico será desarrollar un prototipo de integración y gestión de servicios universitarios, preparado para conectarse posteriormente con sistemas institucionales mediante APIs o mecanismos de interoperabilidad autorizados.

2. SITUACIÓN PROBLEMÁTICA

En una institución de educación superior convergen diariamente diferentes procesos y actores: estudiantes, docentes, coordinadores, personal administrativo, directivos y diferentes dependencias académicas y administrativas.

Cada uno de estos actores requiere consultar información o realizar diferentes solicitudes. En un entorno universitario, la existencia de múltiples servicios digitales puede generar la necesidad de contar con mecanismos que faciliten al usuario identificar, consultar y gestionar sus diferentes requerimientos desde una experiencia integrada.

Por ejemplo, un estudiante puede necesitar consultar información académica, solicitar un recurso, realizar una reserva, registrar una petición, consultar una notificación o conocer el estado de una solicitud.

Desde una perspectiva tecnológica, estos procesos representan una oportunidad para estudiar cómo diferentes servicios pueden comunicarse



entre sí mediante APIs, servicios web y arquitecturas distribuidas.

Por esta razón, se propone construir una plataforma denominada UAJS Smart Campus, que permita centralizar la experiencia del usuario y, al mismo tiempo, mantener los diferentes procesos organizados como componentes independientes.

3. PROBLEMA

¿Cómo diseñar e implementar una plataforma web que permita articular diferentes servicios universitarios mediante una arquitectura modular y distribuida, facilitando la gestión de solicitudes, recursos, comunicaciones y consultas de la comunidad académica de la Universidad Antonio José de Sucre?

4. JUSTIFICACIÓN

El proyecto permitirá desarrollar una solución basada en una problemática relacionada directamente con el contexto universitario y regional de Sucre.

Desde el punto de vista académico, permitirá que los estudiantes integren conocimientos de desarrollo web, arquitectura MVC, programación frontend, programación backend, bases de datos, APIs REST, servicios web, autenticación, comunicación entre sistemas y arquitecturas distribuidas.

La propuesta se encuentra alineada con el propósito de la asignatura de diseñar sistemas distribuidos seleccionando tecnologías, arquitecturas y estándares adecuados para resolver problemas del mundo real.

5. OBJETIVO GENERAL

Desarrollar un prototipo de plataforma web distribuida para la articulación y gestión de servicios universitarios de la Universidad Antonio José de Sucre, utilizando una arquitectura MVC, tecnologías web modernas, APIs REST y servicios independientes que permitan demostrar principios de comunicación, escalabilidad y distribución de componentes.

6. OBJETIVOS ESPECÍFICOS

1. Analizar las necesidades de interacción de los diferentes actores de la comunidad universitaria.
2. Identificar los principales servicios que podrían ser articulados dentro de una plataforma universitaria.
3. Diseñar la arquitectura general de UAJS Smart Campus.
4. Desarrollar el frontend utilizando HTML5, CSS3, JavaScript y React.
5. Implementar el patrón Modelo-Vista-Controlador.
6. Desarrollar un backend mediante Node.js y Express o una tecnología equivalente.
7. Implementar APIs REST para la comunicación entre los diferentes componentes.
8. Diseñar una base de datos para gestionar usuarios, solicitudes, reservas, recursos y notificaciones.

9. Implementar mecanismos de autenticación y autorización.
10. Diseñar una arquitectura que permita separar diferentes funcionalidades en servicios independientes.
11. Analizar mecanismos para garantizar disponibilidad, escalabilidad y tolerancia a fallos.
12. Proponer mecanismos de integración futura con sistemas institucionales existentes mediante APIs o mecanismos autorizados.

7. USUARIOS DEL SISTEMA

ESTUDIANTE:

- Consultar su información.
- Consultar servicios disponibles.
- Registrar solicitudes.
- Consultar el estado de sus solicitudes.
- Solicitar recursos.
- Realizar reservas.
- Consultar notificaciones.
- Consultar actividades institucionales.

DOCENTE

- Consultar sus servicios.
- Solicitar recursos.
- Gestionar reservas.
- Consultar solicitudes.



- Publicar o consultar información académica.
- Recibir notificaciones.

ADMINISTRATIVO

- Gestionar solicitudes.
- Administrar recursos.
- Gestionar reservas.
- Atender requerimientos.
- Actualizar estados.
- Generar reportes.

ADMINISTRADOR DEL SISTEMA:

- Administrar usuarios.
- Administrar roles.
- Gestionar servicios.
- Consultar estadísticas.
- Gestionar configuraciones.
- Supervisar los diferentes componentes.

8. MÓDULOS DE LA PLATAFORMA

8.1. AUTENTICACIÓN Y USUARIOS

Inicio de sesión, cierre de sesión, recuperación de acceso, roles, permisos y gestión de usuarios.

8.2. DASHBOARD UNIVERSITARIO

Servicios disponibles, solicitudes pendientes, reservas, notificaciones, eventos y alertas.

8.3. SERVICIO DE SOLICITUDES

Registro y seguimiento de solicitudes universitarias con usuario, tipo, dependencia, fecha, descripción, prioridad, estado y responsable.

Estados: REGISTRADA → EN REVISIÓN → ASIGNADA → EN PROCESO → RESUELTA → CERRADA.

8.4. SERVICIO DE RESERVAS

Gestión de salas, laboratorios, equipos, espacios académicos y recursos tecnológicos, verificando disponibilidad, fecha, hora, usuario y recurso.

8.5. SERVICIO DE RECURSOS

Administración de recursos disponibles, incluyendo código, nombre, tipo, ubicación, estado y disponibilidad.



8.6. SERVICIO DE NOTIFICACIONES:

Generación de notificaciones por cambios de estado, confirmaciones de reservas, alertas y comunicaciones.

8.7. SERVICIO DE EVENTOS Y ACTIVIDADES
Gestión de conferencias, seminarios, talleres, eventos académicos y actividades institucionales.

8.8. SERVICIO DE PQRS:

Módulo académico para peticiones, quejas, reclamos y sugerencias, planteado como componente de integración y no como sustitución del sistema institucional.

9. TECNOLOGÍAS RECOMENDADAS

FRONTEND

- HTML5
- CSS3
- JavaScript ES6+
- React
- React Router
- Axios o Fetch API

BACKEND

- Node.js

- Express.js

BASE DE DATOS

- MySQL

COMUNICACIÓN

- REST
- HTTP/HTTPS
- JSON

SEGURIDAD

- JWT
- Roles
- Permisos
- Middleware de autenticación

HERRAMIENTAS

- Visual Studio Code
- Git
- GitHub
- Postman
- Swagger/OpenAPI

PROFUNDIZACIÓN

- Docker
- API Gateway

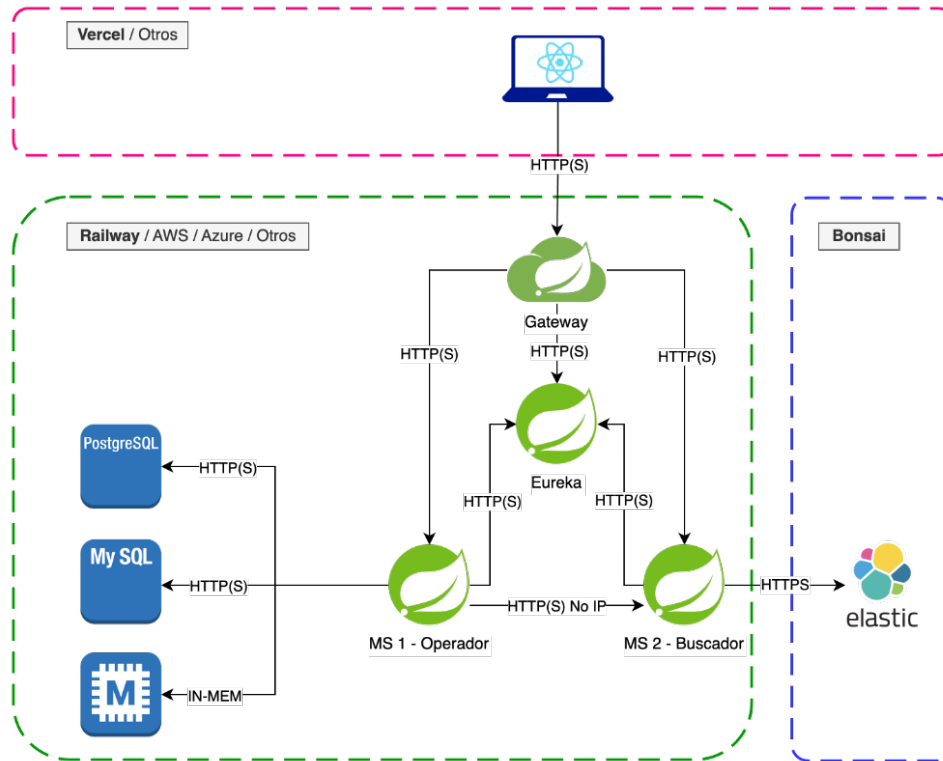
- Redis
- RabbitMQ

ACTIVIDAD INTEGRATIVA PROYECTO DE AULA:

DESARROLLO DE UN *FRONT-END* UTILIZANDO REACT

OBJETIVOS:

- ▶ Desarrollar un *front-end* haciendo uso de HTML5, CSS3, JavaScript y React.
- ▶ Desarrollar un *back-end* haciendo uso de una arquitectura orientada a microservicios, donde cada microservicio expondrá una API REST por medio de Java y Spring.
- ▶ Integrar ambas partes y desplegar el *front-end* y *back-end* tanto de forma local como pública.
- ▶ Hacer uso de bases de datos relacionales y del motor de búsqueda Elasticsearch.



Esta actividad consiste en la realización del **front-end** de la **aplicación web** que se construirá tomando como base el Proyecto Integrador Smart Campus. Por ello, esta actividad permite poner en práctica lo estudiado:

- ▶ Creación de páginas con HTML5.
- ▶ Aplicación de estilos con CSS3 y metodología BEM.
- ▶ Manejo de JavaScript (ECMAScript V6).
- ▶ Uso de la librería React:
 - Orientación a componentes.
 - *Hooks*.
 - React Router V6.

Pautas de elaboración

Esta actividad abarca la creación del **front-end** de una aplicación web haciendo uso de los lenguajes estudiado: HTML, CSS y JavaScript. Además, haremos uso de la librería React.

El *front-end* desarrollado deberá hacer uso de los **principales conceptos** que se han estudiado:

- ▶ Uso de la librería React para construir un frontal que utilice componentes funcionales (**mínimo de diez**) y que haga uso de JSX (por ende, HTML).
- ▶ Uso al menos de los *hooks* estudiados más importantes, `useState` y `useEffect`.
- ▶ Implementación de un *custom hook* a elección.
- ▶ Uso de **React Router** para implementar diferentes rutas en la aplicación (mínimo de cuatro).
- ▶ Aplicar estilos mediante hojas de estilo CSS. Se permite utilizar Bootstrap, Tailwind y SCSS, pero se debe seguir la **metodología BEM**.

La **temática** de la aplicación girará en torno al **proyecto Integrador Smart Campus UNIAJS**. Así pues, la **funcionalidad mínima necesaria** que se debe cubrir es la siguiente:

- **Vista de acceso:** mostrará una landing institucional de **UAJS Smart Campus**, en la que se presentará el nombre de la plataforma, una



breve descripción de sus servicios y los principales perfiles de usuario. Después de cinco segundos, si el usuario no realiza ninguna acción, se podrá redireccionar automáticamente a la página principal de la aplicación.

- **Vista de página principal:** mostrará el panel principal de **UAJS Smart Campus**, donde se podrán visualizar los principales servicios universitarios disponibles, tales como **Solicitudes, Reservas, Recursos, Eventos, Notificaciones y PQRS**. La interfaz deberá contar con una barra de búsqueda que permita localizar servicios o información disponible en la plataforma. Para efectos de esta actividad, la búsqueda podrá realizarse inicialmente sobre los nombres o categorías de los servicios.
- **Vista de servicio:** permitirá seleccionar un servicio universitario y visualizar su información detallada. Por ejemplo, al seleccionar el servicio de **Reservas**, el usuario podrá consultar su descripción, los recursos disponibles y las opciones relacionadas con la solicitud de una reserva. De igual manera, cada servicio deberá presentar las funcionalidades correspondientes a su propósito.
- **Vista de solicitudes:** permitirá al usuario visualizar las solicitudes que haya registrado dentro de la plataforma. Cada solicitud deberá mostrar información como número de solicitud, tipo de servicio,

fecha, estado y descripción. El usuario podrá consultar el detalle de una solicitud y realizar seguimiento a su estado.

- **Vista de reservas:** permitirá consultar los recursos universitarios disponibles para reserva, tales como salas, laboratorios, equipos o espacios académicos. El usuario podrá seleccionar un recurso, consultar su disponibilidad y registrar una solicitud de reserva.
- **Vista de notificaciones:** permitirá visualizar las comunicaciones generadas por la plataforma. Las notificaciones podrán informar sobre cambios en el estado de una solicitud, confirmación o modificación de una reserva, nuevos eventos o cualquier otra información relevante para el usuario.
- **Vista de eventos y actividades:** permitirá consultar las actividades académicas e institucionales disponibles, mostrando información como nombre del evento, fecha, hora, lugar y descripción.
- **Vista de perfil:** permitirá mostrar la información básica del usuario que interactúa con la plataforma, incluyendo nombre, tipo de usuario, programa o dependencia y otros datos definidos para el prototipo.
- **Vista de detalle de solicitud:** permitirá consultar toda la información relacionada con una solicitud específica y visualizar la evolución de su estado. Por ejemplo:



Registrada → En revisión → Asignada → En proceso → Resuelta → Cerrada.

- **Dashboard:** tanto desde la página principal como desde las diferentes áreas de la plataforma, el usuario podrá acceder a un panel de indicadores donde se visualizarán elementos como solicitudes pendientes, reservas realizadas, notificaciones no leídas, eventos próximos y servicios disponibles.

BASE DE DATOS Y GESTIÓN DE INFORMACIÓN

En esta actividad, la aplicación **UAJS Smart Campus** deberá incorporar una **base de datos** que permita almacenar y gestionar la información necesaria para el funcionamiento de la plataforma. A diferencia de una primera aproximación basada únicamente en datos de prueba (*mocks*), en esta etapa el estudiante deberá implementar una estructura de persistencia que permita que la información registrada en la aplicación pueda ser almacenada, consultada, actualizada y eliminada.

La base de datos deberá contener, como mínimo, información relacionada con:

- Usuarios.
- Roles.



- Servicios universitarios.
- Solicitudes.
- Reservas.
- Recursos.
- Eventos y actividades.
- Notificaciones.

Para el desarrollo se recomienda utilizar **MySQL**, aunque podrá utilizarse otro sistema gestor de bases de datos relacional equivalente, siempre que se justifique técnicamente la elección.

La aplicación deberá contar con un **back-end** que permita gestionar la comunicación entre el frontend desarrollado en React y la base de datos. Para ello, se recomienda utilizar **Node.js y Express**, implementando una arquitectura organizada bajo el patrón **Modelo-Vista-Controlador (MVC)**.

La comunicación entre el frontend y el backend deberá realizarse mediante **API REST**, utilizando el protocolo HTTP/HTTPS y el formato **JSON** para el intercambio de información.

La arquitectura básica será:

React → API REST → Backend/MVC → Base de Datos MySQL



Por ejemplo, cuando un estudiante registre una solicitud de servicio, la información deberá ser enviada desde el frontend hacia el backend mediante una petición HTTP. El backend procesará la solicitud y almacenará la información correspondiente en la base de datos.

Posteriormente, cuando el estudiante consulte sus solicitudes, el frontend realizará una petición a la API y el backend recuperará la información almacenada en la base de datos para devolverla en formato JSON.

Dado que esta actividad pertenece a la asignatura **Sistemas Distribuidos**, la solución **UAJS Smart Campus** deberá implementar una **arquitectura basada en microservicios**, con el propósito de que los estudiantes apliquen de manera práctica los conceptos relacionados con distribución de componentes, comunicación entre servicios, independencia de procesos, escalabilidad y disponibilidad.

La aplicación no deberá desarrollarse como un único sistema monolítico. Las principales funcionalidades deberán organizarse en **servicios independientes**, donde cada microservicio sea responsable de una funcionalidad específica del sistema.

Como mínimo, se recomienda implementar los siguientes microservicios:

- **Microservicio de Usuarios:** gestión de usuarios, roles y autenticación.



- **Microservicio de Solicitudes:** registro, consulta, actualización y seguimiento de solicitudes.
- **Microservicio de Reservas:** gestión de reservas de salas, laboratorios, equipos y otros recursos.
- **Microservicio de Recursos:** administración de los recursos universitarios disponibles.
- **Microservicio de Eventos:** gestión de eventos y actividades académicas o institucionales.
- **Microservicio de Notificaciones:** generación y gestión de notificaciones relacionadas con las operaciones realizadas en los demás servicios.

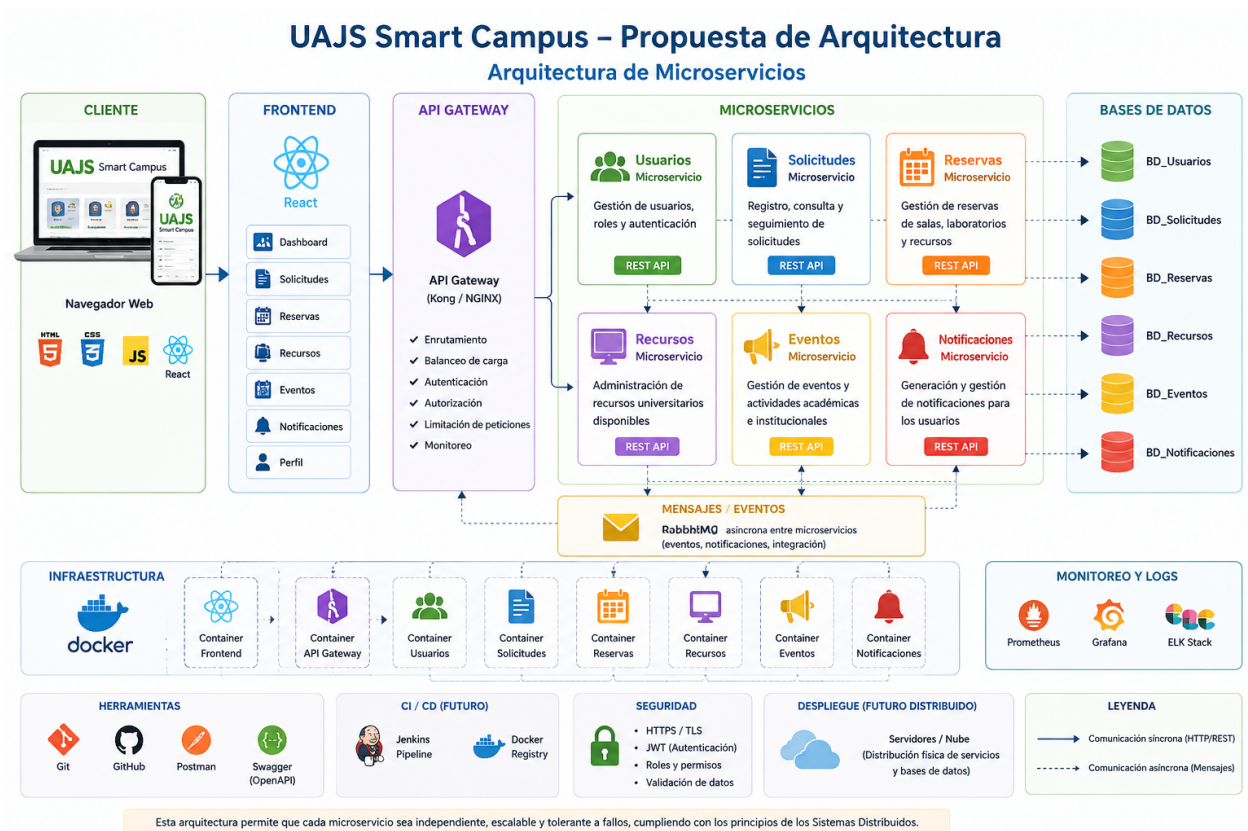
Cada microservicio deberá disponer de su propia lógica de negocio y exponer sus funcionalidades mediante una **API REST**, utilizando HTTP/HTTPS y JSON como mecanismos de comunicación.

La arquitectura general propuesta será:

Frontend React → API Gateway → Microservicios → Bases de datos

Los estudiantes deberán demostrar, como mínimo, operaciones **CRUD (Create, Read, Update y Delete)** sobre los principales recursos de la plataforma y evidenciar la comunicación correcta entre:

Frontend React → API REST → Backend → Base de Datos.



La información utilizada durante el desarrollo deberá corresponder a **datos ficticios o de prueba**. No se deberán utilizar datos personales reales de estudiantes, docentes o funcionarios de la Universidad Antonio José de Sucre sin la correspondiente autorización institucional.

La entrega consistirá **un único archivo ZIP** con los siguientes archivos:



- ▶ **Videomemoria obligatoria** en formato MP4 de la actividad.
- ▶ Archivo TXT con la **URL del proyecto desplegado** en Vercel u otro.
- ▶ **Proyecto con el código** de la aplicación sin `node_modules`.

¿Qué es una vídeo memoria?

La vídeo memoria es una grabación donde explicarás con detalle el *software* que has desarrollado. Si estás muy orgulloso del *software* que has construido, ¡podrías incluso publicar este vídeo de forma pública como parte de tu portfolio!

En esta ocasión, la videomemoria tendrá una duración máxima de **diez minutos** y deberá visitar los siguientes aspectos de tu actividad:

- ▶ **Componentes de React:** se listarán y explicarán brevemente todos los componentes de React que forman parte de la solución desarrollada. Hablarás aquí también del enrutador que has utilizado.
- ▶ **Hooks utilizados:** se listarán todos los *hooks* utilizados en la solución desarrollada y en qué componentes. Además, se incluirán aquí los *hooks* personalizados que se hayan creado, indicando por qué son necesarios y el valor que aportan a la solución.
- ▶ **Vistas:** mostrarás cómo el proyecto puede arrancarse de forma local. Una vez desplegado en local, se mostrarán las diferentes vistas del *front-end* desarrollado, junto con una descripción breve de su contenido que

haga énfasis en los estilos CSS que se utilizaron (los más destacables, no es necesario hablar de todo el código CSS).

- **Despliegue:** se mostrará el proyecto desplegado en Vercel correctamente.

Rúbrica

Criterio	Descripción	Puntuación máxima (puntos)	Peso %
Criterio 1	Implementación de componentes funcionales en React (mínimo de diez) haciendo uso de JSX.	0,5	5 %
Criterio 2	Uso de los <i>hooks</i> principales de React: <i>useState</i> y <i>useEffect</i> .	0,5	5 %
Criterio 3	Implementación de un <i>custom hook</i> que sea utilizado en algún componente funcional.	1	10 %
Criterio 4	Uso correcto de React Router para crear diferentes rutas dentro de la aplicación.	1	10 %
Criterio 5	Modificación del estilo de la aplicación mediante el uso de CSS. Uso de metodología BEM.	1	10 %
Criterio 6	Implementación y funcionamiento correcto de las vistas necesarias para cubrir la funcionalidad mínima de la aplicación.	4,5	45 %
Criterio 7	Elaboración de la videomemoria obligatoria según las especificaciones, ajustándose a la duración aproximada y	1	10 %

	cubriendo los aspectos solicitados.		
Criterio 8	Despliegue en Vercel	0,5	5 %
		10	100 %